👌 XAI: Chess Knowledge in AlphaZero & Interpreting Stable Diffusion

FAMAF - UNC

First Semester 2026

Disclaimer ---

This course is based on

Explainable Artificial Intelligence

From Simple Predictors to Complex Generative Models

Spring 2023, Harvard University

https://interpretable-ml-class.github.io/

Paper 1 ---

Acquisition of Chess Knowledge in AlphaZero

Thomas McGrath[1,+], Andrei Kapishnikov[2,+], Nenad Tomašev[1], Adam Pearce[2], Demis Hassabis[1], Been Kim[2], Ulrich Paquet[1], and Vladimir Kramnik[3]

[1]DeepMind
[2]Google Brain
[3]World Chess Champion, 2000–2007*
[+]these authors contributed equally to this work

ABSTRACT

What is learned by sophisticated neural network agents such as AlphaZero? This question is of both scientific and practical interest. If the representations of strong neural networks bear no resemblance to human concepts, our ability to understand faithful explanations of their decisions will be restricted, ultimately limiting what we can achieve with neural network interpretability. In this work we provide evidence that human knowledge is acquired by the AlphaZero neural network as it trains on the game of chess. By probing for a broad range of human chess concepts we show when and where these concepts are represented in the AlphaZero network. We also provide a behavioural analysis focusing on opening play, including qualitative analysis from chess Grandmaster Vladimir Kramnik. Finally, we carry out a preliminary investigation looking at the low-level details of AlphaZero's representations, and make the resulting behavioural and representational analyses available online.

1 Introduction

(McGrath et al. 2022)

Learning from machines (AlphaZero) ---

- Most methods we have looked so far try to interpret algorithms trained on human-generated data and labels
  - These interpretations may resemble human-understandable representations only because they learned from such data
- Can we interpret what the algorithms has been learning through its self-play training process?

Learning from Machines: Three-Pronged Acquisition ---

- **Probe for concepts**
  - How closely is AlphaZero's internal representation **related to** chess concepts humans have already created?
  - Detection of human concepts from network activations
- **Study behavioral changes**
  - How do changing representations give rise to changing behaviors?
  - Evolution of AlphaZero vs. human's strategy in openings
- **Investigate activations directly**
  - Unsupervised methods using non-negative matrix factorization (NMF) and direct measure of covariance to discover concepts not represented by humans in the past

Interpretability ---

Concept-based (post-hoc) interpretability

- **Network probing:** detect human concepts from internal activations using linear classifiers (*concept activation vectors*)
- **Important features:** which concepts matter most for model predictions?
- **Mechanistic understanding:** what algorithms do individual layers implement?
- **Key challenge:** probing measures *correlation*, not *causation* — a concept linearly decodable from activations may not causally drive behavior

Explainability in reinforcement learning

- **Structural causal models:** simulate interventions on actions to answer counterfactual questions (*what if a different move was played?*)
- **Reward difference explanations:** why was action a chosen over a'? Quantify the expected reward gap
- **Behavioral trajectory analysis:** identify critical decision points where the agent's policy changes most dramatically

Chess as Testing Ground for AI Interpretability ---

Can humans learn the machine's strategy?

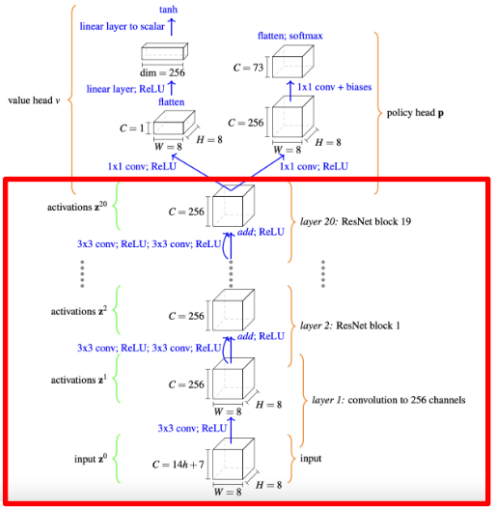
- Does not rely on human-labeled data
- Tree-based organization / saliency maps
- Natural language processing to generate move-by-move commentary
- **This paper:** captures "intuitive" aspect of chess play by understanding networks that produce value assessment (**v**) and candidate move (**p**)

$$\mathbf{p},\ v = f_\theta(\mathbf{z}^0)$$

In plain English

$\mathbf{z}^0$ is the board position fed to the network; $\mathbf{p}$ is a probability distribution over legal moves (what AlphaZero *wants* to play), and $v \in [-1, 1]$ is its estimate of who is winning. The question of the paper is: *what does f_θ know about chess to produce these outputs?*

AlphaZero: Network Structure and Training ---

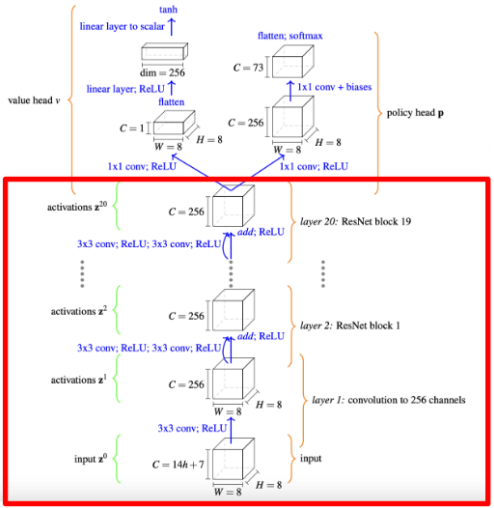


$$\mathbf{p},\ v = f_\theta(\mathbf{z}^0)$$

$$\mathbf{z}^l = f^l(\mathbf{z}^{l-1}) = \mathrm{ReLU}\Big(\mathbf{z}^{l-1} + g^l(\mathbf{z}^{l-1})\Big)$$

$$\mathbf{z}^l = f^{1:l}(\mathbf{z}^0) = f^l \circ \cdots \circ f^2 \circ f^1(\mathbf{z}^0)$$

AlphaZero: Network Structure and Training ---



In plain English

In the diagram, each box in the red rectangle is one residual block. The **"add; ReLU"** arrow inside each box is where the network adds the unmodified input to the learned transformation — instead of replacing it. This repeats 20 times, building progressively more abstract representations of the board.

Probing for Concepts - Encoding of Human Conceptual Knowledge ---

Question: Can human concepts be easily predicted from the network's internal representation?

Concept: User-defined function mapping network input to real line:

$$c(\mathbf{z}^0) = \begin{cases} 1 & \text{if } \mathbf{z}^0 \text{ contains a bishop-pair U+2657 for the playing side} \\ 0 & \text{otherwise} \end{cases}$$

Approach: Train a sparse linear regression model from activations $\mathbf{z}^l$ at layer l and training step t to human concept j

In plain English

A *concept* is any chess idea a human can define on a board position — from simple material facts (does the player have the bishop pair?) to complex positional ideas (is the king safe?). If a simple linear model can predict c_j from $\mathbf{z}^l$, the concept is **linearly encoded** in

Key Insight

The network was never explicitly trained to represent chess concepts — it only learned to predict moves and outcomes via self-play. Finding linearly decodable concepts means AlphaZero **spontaneously developed human-like internal representations**.

Probing Concept Learning: Details ---

- **Data:** Randomly sample training, validation, test data from ChessBase archive, compute concept values and AlphaZero activations
- **Procedure:** For concept i, layer l, training step t, solve:

$$\mathbf{w}_{jlt},\ b_{jlt} = \min_{\mathbf{w},b} \frac{1}{N} \left\| \mathbf{w}^T \mathbf{Z}_t^l + b\mathbf{1} - \mathbf{c}_j \right\|_2^2 + \lambda \|\mathbf{w}\|_1 + \lambda |b|$$

- **Controls:** Regression from $\mathbf{z}^0$ and random concept regression
- **Evaluation:** R^2 value, fraction of variance in concept explained by network activation

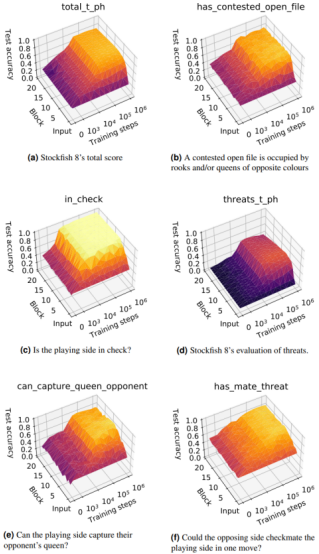
In plain English

Fit a Lasso regression: predict concept c_j from activations $\mathbf{z}^l$. The ℓ_1 penalty keeps only the **few neurons** that actually encode the concept.

Key Insight

High R^2 = the concept is **linearly readable** from that layer. Repeat for every layer l and checkpoint $t \rightarrow$ a map of *what, when, and where* each concept is learned.

Evolution of Human Concepts in AlphaZero 1/2 ---



(a) Stockfish 8's total score

(b) A contested open file is occupied by rooks and/or queens of opposite colours

(c) Is the playing side in check?

(d) Stockfish 8's evaluation of threats.

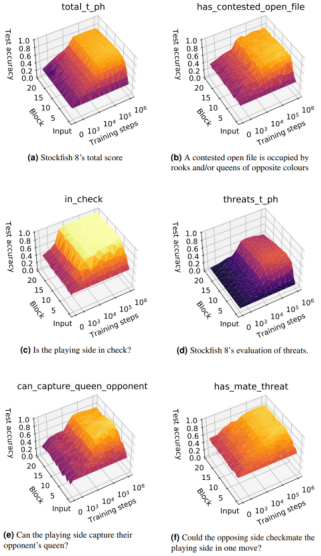
(e) Can the playing side capture their opponent's queen?

(f) Could the opposing side checkmate the playing side in one move?

Each surface is a *what–when–where* plot: probe R^2 as a function of **network depth** (Block) and **training time** (Training steps).

- **x-axis:** training steps ($10^0 \rightarrow 10^6$) — *when* is it learned?
- **y-axis:** block index (1–20) — *where* in the network?
- **z-axis:** test accuracy — *how well* is the concept encoded?

Evolution of Human Concepts in AlphaZero 2/2 ---



(a) Stockfish 8's total score

(b) A contested open file is occupied by rooks and/or queens of opposite colours

(c) Is the playing side in check?

(d) Stockfish 8's evaluation of threats.

(e) Can the playing side capture their opponent's queen?

(f) Could the opposing side checkmate the playing side in one move?

- A surface that rises early and stays high means AlphaZero learned that concept **fast and robustly** (e.g. `in_check`).
- A surface that stays flat means the concept is **never linearly encoded** (e.g. `threats_t_ph` — still dark/purple throughout).
- Simple tactical concepts (`in_check`, `can_capture_queen`) emerge **early and in middle layers**. Complex strategic concepts (`total_t_ph`, `has_mate_threat`) require **more training steps** and tend to concentrate in **deeper layers** — mirroring how humans learn chess: tactics before strategy.

Key Findings -- ---

1. **Grokking:** Most concepts emerge abruptly around **32,000 training steps** accuracy is near zero before, then jumps and stabilizes. AlphaZero does not learn gradually; it *clicks*.
2. **Information drop in deep layers:** for some concepts, probe accuracy *decreases* in the last blocks — the network re-encodes information in a less linearly-separable form as it approaches the output heads.
3. **Highly-distributed representations:** some concepts **cannot** be probed — sparsity forces the probe to use few neurons, but if the concept is spread across many neurons simultaneously, a sparse linear probe cannot recover it.
4. **Learning from errors:** positions where the probe fails systematically may reflect a genuine **"difference of opinion"** between AlphaZero and Stockfish — not a probe failure, but AlphaZero evaluating the position by a different (possibly superior) criterion.

Key Insight

Findings 3 and 4 are honest about the method's limits: absence of a detectable concept $\neq$ absence of the concept — it may simply be encoded in a way a **linear probe cannot see**.

Challenges for Concept Probing ---

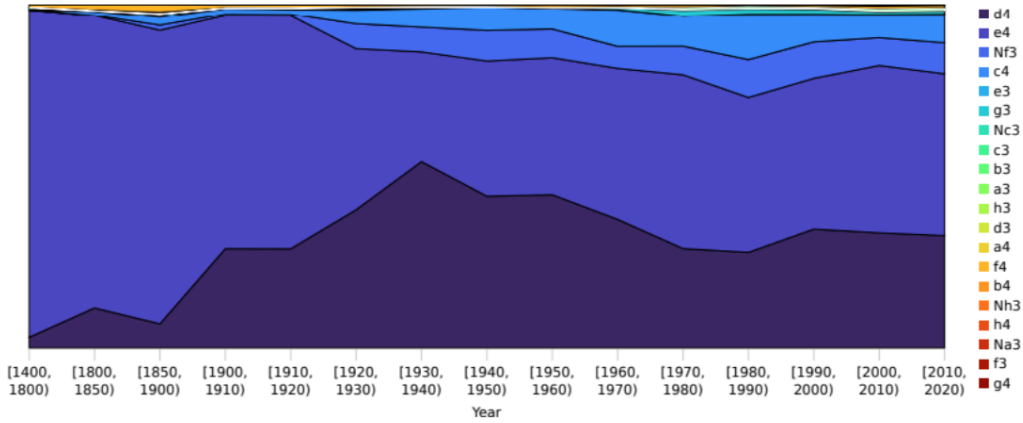
1. What's the right **probing architecture**?
2. How should we interpret **complex or subjective concepts**?
3. When can we definitively say a **concept is represented**?
4. When we train a probe, we cannot tell if we are getting a **confounder or the concept** itself

Progression through AlphaZero and Human History

Progression of Knowledge ---

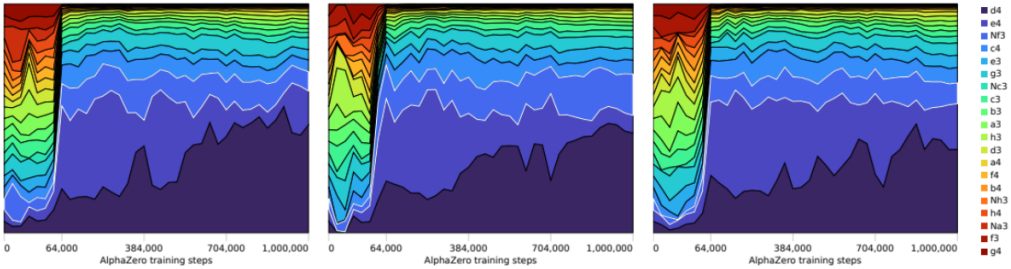
- Recall: $\mathbf{p},\ v = f_\theta(\mathbf{z}^0)$
- **Question:** How does the progression of AlphaZero's knowledge compare to that of humans?
- **Key Findings:**
  - AlphaZero: Start from a uniform prior, then narrow down.
  - Humans: Start from a concentrated prior, then expand.

Progression through Human History ---



Each color represents a first move (e.g. e4, d4, Nf3), with its area showing how often humans played it across history — serving as a reference to compare whether AlphaZero's training recapitulates or diverges from human chess evolution.

Progression through AlphaZero History ---



The three panels show **three independent AlphaZero training runs**: the distribution is remarkably consistent across runs — AlphaZero converges to similar move preferences regardless of random initialization. Before 64,000 steps, chaotic exploration dominates; after that, the distribution stabilizes abruptly (**grokking**). Compared to human history, AlphaZero settles on a mix of `d4` and `e4` but with a more diverse and volatile distribution — **it does not recapitulate human chess history**, but finds its own path.

Progression of AlphaZero's Chess Knowledge ---

Methodology: At different training checkpoints (up to 128k steps), examine AlphaZero's move tendencies and concept probe accuracy.

Primary Takeaways

1. *AlphaZero learns standard opening theory* **early** — *move preferences stabilize and match known theory within the first 64k steps*
2. *AlphaZero learns* **material values before positional concepts** — *piece counts are encoded first; king safety and mobility come later*

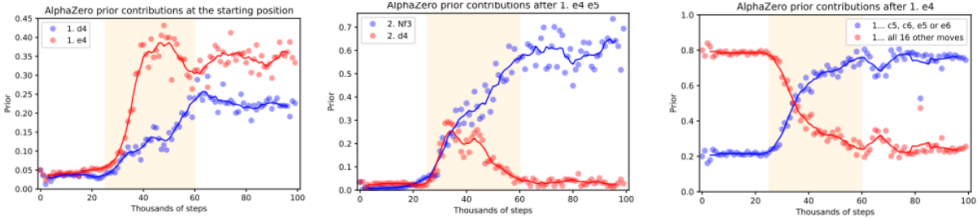
Both findings reinforce that AlphaZero learns basic human chess concepts first

Key Insight

This mirrors the curriculum of a human chess student: openings and material counting are taught before strategic positional play. AlphaZero rediscovers this ordering **from scratch**, via self-play alone — with no exposure to human games or pedagogy.

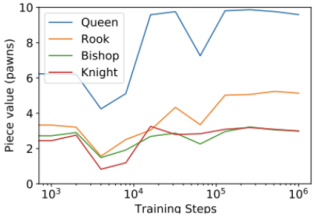
Opening Theory Knowledge ---

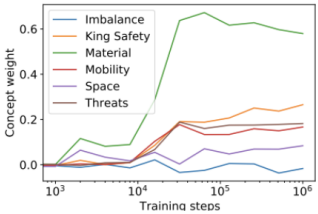
1. ∼30–60k: AlphaZero plays *1. e4/d4* the majority of the time
2. ∼45k: AlphaZero considers *2. d4* before opting for *2. Nf3*
3. ∼45k: AlphaZero plays standard responses to *1. e4*



- **Graph 1:** What move should White play to open the game?
- **Graph 2:** What should White play on move 2, after Black responds with e5?
- **Graph 3:** How should Black respond to White's opening move e4?

Material vs. Positional Knowledge ---





Top — Material:

The *relative value of pieces*, measured in pawns. Standard human values are: Queen=9, Rook=5, Bishop=3, Knight=3. AlphaZero independently rediscovers approximately these values by ~100k steps.

Bottom — Positional concepts:

- **Material:** piece count advantage
- **King Safety:** exposure to attacks
- **Mobility:** number of legal moves available
- **Space:** board control
- **Threats:** can capture next move
- **Imbalance:** asymmetric piece differences (e.g. bishop vs. knight)

The story they tell together: AlphaZero first learns to count pieces (material, ~30k steps), and only later develops subtler positional concepts like king safety and mobility — exactly the order in which a human learns chess.

Training Progression Assessment: GM Vladimir Kramnik ---

Qualitative evaluation by World Chess Champion Vladimir Kramnik, who analyzed AlphaZero's play at three training checkpoints:

1. **16k to 32k steps:** understands material value in complex positions — knows which pieces are worth sacrificing and which are not
2. **32k to 64k steps:** develops king safety awareness in imbalanced positions — starts recognizing when the king is genuinely at risk
3. **64k to 128k steps:** combines king safety with material sacrifices in complex positions — a hallmark of grandmaster-level strategic play

Key Insight:

- Tactical skills appear to precede positional skills as AlphaZero learns
- This is a **human expert's qualitative confirmation** of what the concept probes showed quantitatively: AlphaZero's learning curriculum — material first, positional concepts later — mirrors how chess mastery develops in humans.

Exploring Additional Feature Detectors in AlphaZero Network

Exploring Activations with Unsupervised Methods ---

Goal: Find feature detectors embedded within the network — *without* using predefined human concepts as supervision.

In plain English

Unlike probing (which asks *"is concept X encoded here?"*), these methods ask *"what patterns exist here?"* with no human concept assumed in advance.

Key Insight

If unsupervised methods independently recover human-like concepts, it strengthens the case that AlphaZero's representations are genuinely structured around chess knowledge — not an artifact of the linear probe design.

Unsupervised Methods: Details ---

a) **Non-Negative Matrix Factorization (NMF):** decompose each layer's activations into a small set of additive factors — each factor represents a pattern of co-activating channels across board positions

b) **Input-activation correlation:** measure the covariance between each channel's activation and the raw input board — reveals which input features drive individual neurons

Primary Takeaway

Individual layers and channels encode *feature detectors* related to human-recognizable chess concepts — discovered **without any concept labels**.

Approach #1: Non-Negative Matrix Factorization ---

Idea: compress each layer's activations into $K < C$ interpretable factors, then visualize each factor on the board to find *feature detectors*.

Step 1: Stack all activations for layer l with C channels: $\hat{\mathbf{Z}}^l \in \mathbb{R}^{NHW \times C}$

Step 2: Factorize $\hat{\mathbf{Z}}^l \approx \cdot_{\text{all}}\mathbf{F}$:

$$\mathbf{F}^*,\ \cdot^*_{\text{all}} = \min_{\mathbf{F},\ \cdot_{\text{all}}} \left\| \hat{\mathbf{Z}}^l - \cdot_{\text{all}}\mathbf{F} \right\|_2^2 \qquad \mathbf{F},\ \cdot_{\text{all}} \geq 0$$

- $\cdot_{\text{all}} \in \mathbb{R}^{NHW \times K}$ — factor scores per board square
- $\mathbf{F} \in \mathbb{R}^{K \times C}$ — which channels compose each factor

Step 3: For each factor k and input n, visualize $\cdot_k$ on the board.

Approach #1: Results ---

In plain English

Each factor k groups channels that fire together. Plotting its scores on the board reveals *where* that pattern activates — if it traces diagonals or controlled squares, the network learned that concept without being told to.

Key Insight

These patterns emerge with **no concept labels** — NMF discovers structure purely from activation co-occurrence. If the recovered factors resemble human chess concepts, it independently confirms the probing results.

Results: NN Matrix Factorization Analysis ---

Diagonal Moves (1st Layer)

Diagonal Moves (2nd Layer)

# of Pieces that could move to each square

Approach #2: Input-Activation Covariance Analysis ---

Idea: for each channel i in layer l, measure how strongly its activation correlates with each input feature — then visualize on the board.

Step 1: Compute covariance between channel i's activation z_i^l and the raw input board $\mathbf{z}^0$:

$$\text{cov}(z_i^l, \mathbf{z}^0) = \mathbb{E}\left[z_i^l \mathbf{z}^0\right] - \mathbb{E}\left[z_i^l\right] \mathbb{E}\left[\mathbf{z}^0\right]$$

Step 2: Visualize $\text{cov}(z_i^l, \mathbf{z}^0)$ on the board to find *feature detectors*.

In plain English

If a neuron fires whenever there is a bishop on a particular diagonal, its covariance with those input squares will be high — the map reveals *what the neuron is looking at* in the input.

Key Insight

Unlike NMF (which finds group patterns across channels), this method zooms into **individual neurons** — asking *what specific input configuration drives this unit?*

Results: Input-Activation Covariance Analysis ---

Detecting Move-Types from a Square:

- 1–2) Diagonal-Attacking Pieces (Queen, Bishop)
- 3–4) Horizontally-Attacking Pieces (Queen, Rook)
- 5) Both

5 covariances with different channels for the square (5, 4) E4

Conclusion: Key Findings (Paper 1) ---

1. Human-Defined Concepts can be regressed from the AlphaZero network
   - Despite never being trained on a human game of chess
2. As training progresses, AlphaZero understands basic concepts (openings, material) before more complex ones (king safety, mobility)
3. Feature Detectors of human-recognizable chess concepts are encoded by individual layers + channels in AZ Network
   - Can be found using supervised & unsupervised techniques

Limitations ---

1. Challenges for Concept Probing (previous section)
2. Knowledge acquisition is not complete — only a very small part of the model
3. Interpreting "Feature Detectors'' found via Unsupervised techniques
   - Inherently subjective
4. No Causal Insight into the Learned Concepts (only correlation)

Future Research Areas ---

1. Addressing Concept Probing Limitations (more than sparse LR)
2. Can we go beyond finding human knowledge embedded in AlphaZero and understand what new concepts are learned?
   - Further analysis of feature detectors found with unsupervised techniques
3. How do we generalize these findings to other machine learning settings? Could we find human-recognizable concepts in different trained models?

Discussion (Paper 1) ---

- How is this paper different from methods we discussed so far?
  - In terms of goals / techniques / specific models to explain
- Does this approach have potential applications for a more general setting other than board games?
  - What are the pros/cons of using games/artificial environments as baseline for evaluating RL algorithms?
- Can this be applied to our practice of doing science?
  - Can we recreate or extract theories using similar frameworks (AI for science)?
- How should we define or understand "knowledge'' in these settings?
- How convinced are you about the "interpretability'' aspect?
  - Who would be potential audience that could benefit from such analysis?

What the DAAM: Interpreting Stable Diffusion Using Cross Attention

Raphael Tang,[*1] Linqing Liu,[*2] Akshat Pandey,[1] Zhiying Jiang,[3] Gefei Yang,[1]
Karun Kumar,[1] Pontus Stenetorp,[2] Jimmy Lin,[3] Ferhan Ture[1]
[1]Comcast Applied AI [2]University College London [3]University of Waterloo
[1]{raphael_tang,akshat_pandey,gefei_yang,karun_kumar,ferhan_ture}@comcast.com
[2]{linqing.liu,p.stenetorp}@cs.ucl.ac.uk [3]{zhiying.jiang,jimmylin}@uwaterloo.ca

Abstract

Large-scale diffusion neural networks represent a substantial milestone in text-to-image generation, but they remain poorly understood, lacking interpretability analyses. In this paper, we perform a text–image attribution analysis on Stable Diffusion, a recently open-sourced model. To produce pixel-level attribution maps, we upscale and aggregate cross-attention word–pixel scores in the denoising subnetwork, naming our method DAAM. We evaluate its correctness by testing its semantic segmentation ability on nouns, as well as its generalized attribution quality on all parts

Figure 1: The original synthesized image and three DAAM maps for "monkey," "hat," and "walking," from the prompt, "monkey with hat walking."

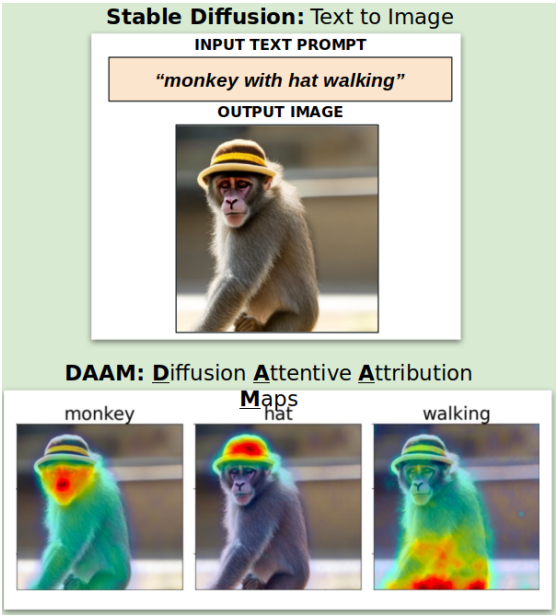
Stability AI recently open-sourced Stable Diffusion (Rombach et al., 2022), a 1.1 billion-parameter latent diffusion model pretrained and fine-tuned on the LAION 5 billion image dataset (Schuhmann

(Tang et al. 2023)

Outline ---

- Related work
- Background: Stable Diffusion
- **DAAM** for text-to-image **attribution**
- Results
  - Attribution quality analysis
  - Syntax to pixels
  - Entanglement
- Limitations & discussion

DAAM estimates per-pixel attribution for each word in a prompt (post-hoc)

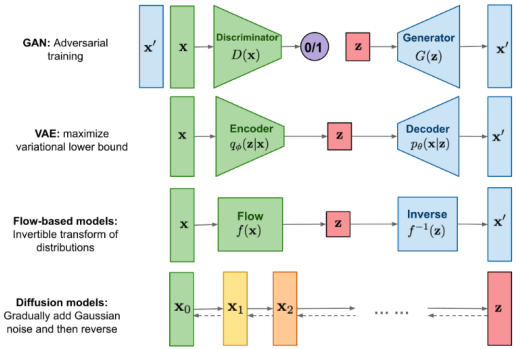


Related Work ---

This work applies existing techniques (cross-attention) to an **open source, SOTA** diffusion model to probe limitations

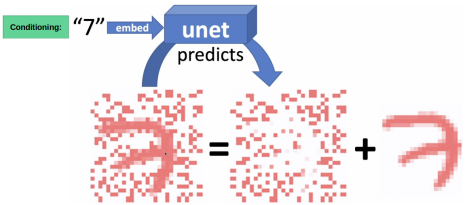
- **Textual perturbation** (Wallace et al., 2019), Attentional Visualization, information bottlenecks to relate important input tokens to outputs of large LMs
- Probing **vision transformers for verb understanding** (Hendricks & Nematzadeh, 2021)
- Enhancing diffusion models using **prompt engineering** (Hertz et al., 2020; Woolf, 2020)
- **Disentangling** e.g., style and spelling (Karras et al., 2019; Materzynska et al., 2022)
- Counterfactual explanations for *text classification* (Jacovi et al., 2021) — unclear how to extend to image generation

Background: Generative Models ---



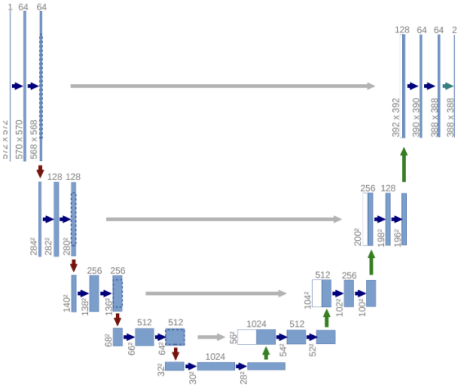
- **GAN:** a discriminator and generator compete — the generator learns to produce images indistinguishable from real ones.
- **VAE:** an encoder maps the image to a latent vector $\mathbf{z}$, a decoder reconstructs it — new images are generated by sampling $\mathbf{z}$.
- **Flow-based:** an invertible transformation f maps images to noise — generation runs f^{-1} with no information loss.
- **Diffusion:** gradually destroys an image with Gaussian noise, then learns to reverse the process step by step from pure noise.

Background: Image Generation with Diffusion ---



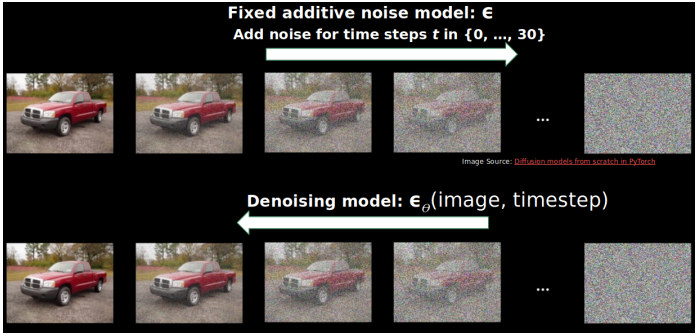
The U-Net receives a noisy image and a text condition (e.g. “7”), predicts the noise component, and subtracts it — leaving a slightly cleaner image. Repeating this process across T steps recovers a sharp image from pure noise. DAAM operates inside this loop, reading the cross-attention scores at each step to measure how much each word influences each image region.

Background: UNet Architecture ---



- **Downsampling (left):** progressively reduces spatial resolution while increasing channels — extracts increasingly abstract features
- **Bottleneck (bottom):** lowest resolution, highest abstraction — where global context is processed
- **Upsampling (right):** progressively restores spatial resolution — reconstructs the image
- **Skip connections (gray arrows):** pass fine-grained spatial detail from encoder to decoder at each resolution level
- **Cross-attention layers:** embedded at each level — this is where DAAM reads word–pixel scores to build attribution maps

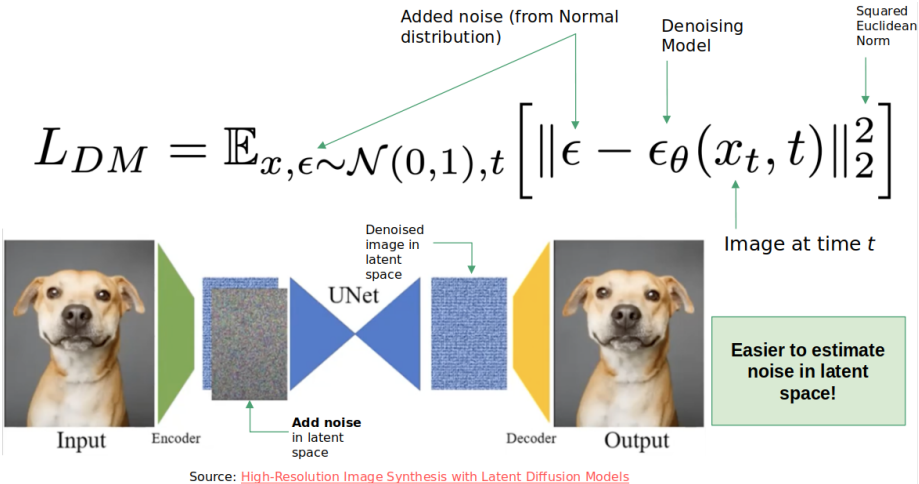
Background: Forward and Reverse Process ---



- **Forward:** a fixed process adds Gaussian noise step by step until the image is destroyed.
- **Reverse:** a network ϵ_θ predicts and subtracts the noise at each step — repeating this reconstructs a sharp image from pure noise.

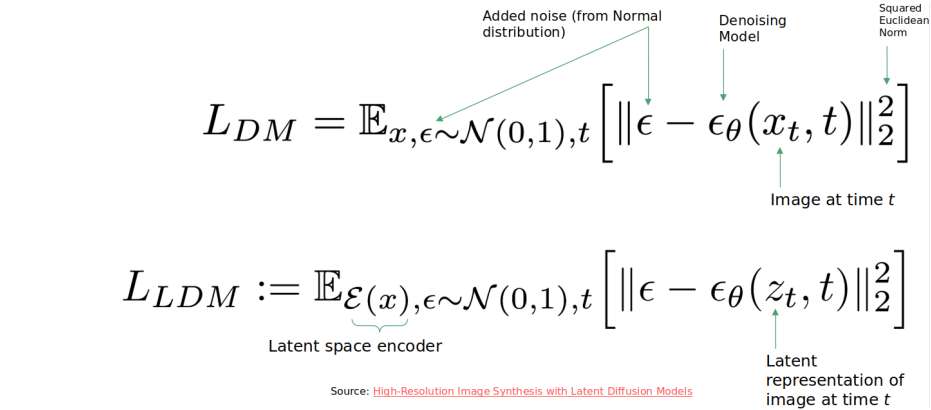
The key point: the network does not learn to generate images directly, but to predict the noise — a regression task that is much more stable to train.

Diffusion Model Loss ---



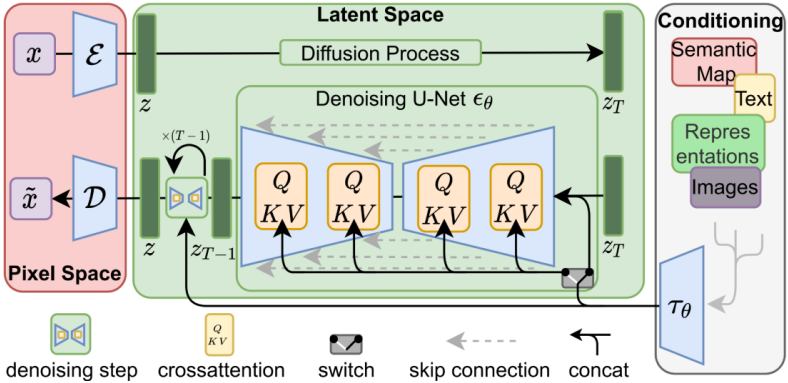
Source: High-Resolution Image Synthesis with Latent Diffusion Models

Latent Diffusion Model Loss ---



Both losses train ϵ_θ to predict the added noise ϵ via squared error. L_{LDM} differs from L_{DM} in one way: it first compresses the image into a latent vector $z_t = \mathcal{E}(x)$, making denoising computationally cheaper — this is the key idea behind Stable Diffusion.

Stable Diffusion Architecture ---



Stable Diffusion conditions the denoising process on text by encoding the prompt into word vectors using CLIP — a model pretrained to align text and images in a shared latent space. These vectors are injected into the U-Net via **cross-attention** at every layer, steering noise removal toward regions that are semantically consistent with each word in the prompt.

Cross-Attention for Text Conditioning ---

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) \cdot V$$

UNet block i — Text encoder — Text encoder

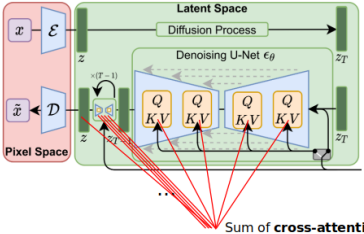
$$Q = W_Q^{(i)} \cdot \varphi_i(z_t), \quad K = W_K^{(i)} \cdot \tau_\theta(y), \quad V = W_V^{(i)} \cdot \tau_\theta(y)$$

Queries: image tokens

Keys, Values: prompt tokens

Cross-attention links image regions to prompt words: **queries** come from the U-Net's spatial representations, while **keys and values** come from the CLIP-encoded prompt. The softmax scores measure how much each image region attends to each word — and these are precisely the scores DAAM aggregates across layers and timesteps to produce per-word attribution maps.

DAAM: Per-Word Heatmaps ---



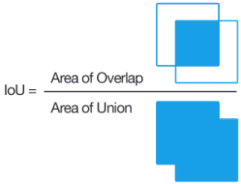
Overlay attention maps on generated image

Results: Attribution Analysis Part 1 -- Object Attribution ---

We can evaluate DAAM as an image-segmentation tool

DAAM Segments Stable Diffusion Images ---

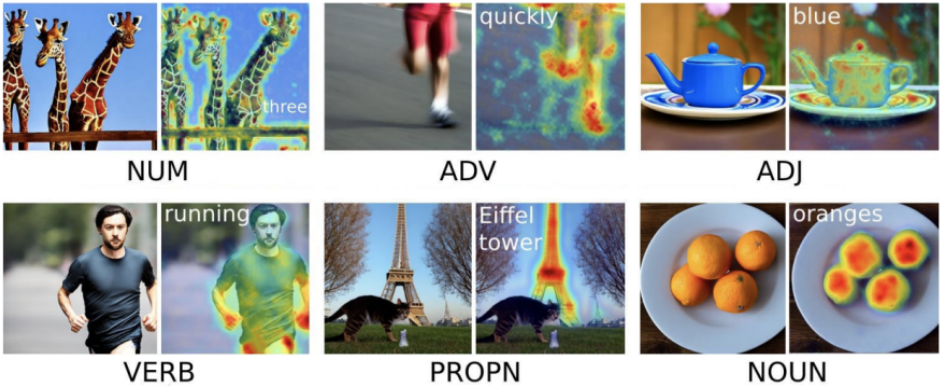
#	Method	COCO-Gen mIoU^{80}	COCO-Gen mIoU$^{\infty}$	Unreal-Gen mIoU^{80}	Unreal-Gen mIoU$^{\infty}$
	Supervised Methods				
1	Mask R-CNN (ResNet-101)	82.9	32.1	76.4	31.2
2	QueryInst (ResNet-101-FPN)	80.8	31.3	78.3	35.0
3	Mask2Former (Swin-S)	**84.0**	32.5	**80.0**	36.7
4	CLIPSeg	78.6	**71.6**	74.6	**70.9**
	Unsupervised Methods				
5	Whole image mask	20.4	21.1	19.5	19.3
6	PiCIE + H	31.3	25.2	34.9	27.8
7	STEGO (DINO ViT-B)	35.8	53.6	42.9	54.5
8	Our DAAM-0.3	64.7	59.1	59.1	**58.9**
9	Our DAAM-0.4	**64.8**	**60.7**	**60.8**	58.3
10	Our DAAM-0.5	59.0	55.4	57.9	52.5



- **Metric:** mIoU — measures overlap between predicted and ground-truth masks; higher is better
- **mIoU^{80}:** restricted to COCO's 80 classes — favors supervised methods trained on them
- **mIoU$^{\infty}$:** open vocabulary — any word in the prompt; favors DAAM
- **Key result:** DAAM outperforms all unsupervised baselines by $\sim$30 points and approaches supervised methods on mIoU$^{\infty}$ — without being trained to segment anything

Results: Attribution Analysis Part 2 -- Generalized Attribution ---

DAAM can segment **beyond nouns**:



Evaluating DAAM with a User Study ---

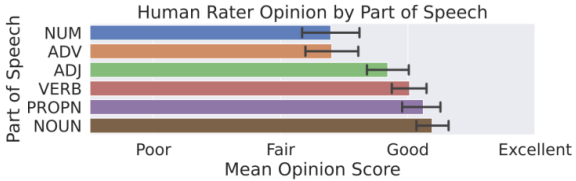
Setup: 50 annotators rated DAAM maps on a 5-point scale (Poor $\rightarrow$ Excellent), with 3 raters per image. Abstract words and poor-quality images were discarded.

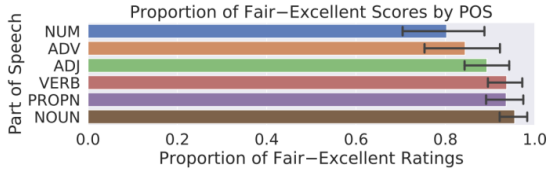
- 50 annotators, none seeing more than 18% of images
- Every image rated by 3 independent annotators
- Abstract words (e.g. "the") and poor images excluded from evaluation

Key Insight

DAAM is evaluated beyond nouns — the study covers all interpretable parts of speech, since words like "running" or "blue" have no ground-truth segmentation mask.

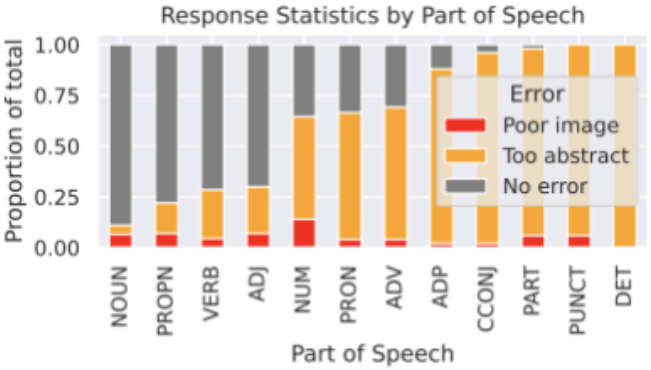
Evaluating DAAM with a User Study Results 1/2 ---





- **Nouns, verbs, adjectives, proper nouns:** mean opinion score close to *Good*
- **Numerals and adverbs:** closer to *Fair* — harder to localize visually
- **All categories:** >80% of ratings are Fair-to-Excellent

Evaluating DAAM with a User Study Results 2/2 ---



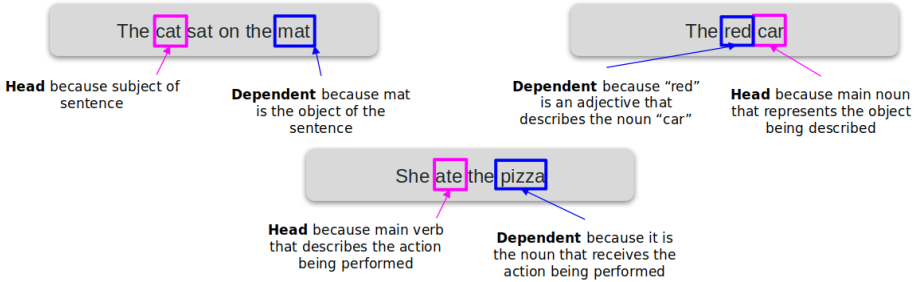
- Lower scores in some categories reflect **word abstractness or generation failures** — not DAAM failures. When the image and word are interpretable,
- DAAM produces plausible attribution maps across all parts of speech.

Key Question

How does **syntax** relate to the generated **pixels**?

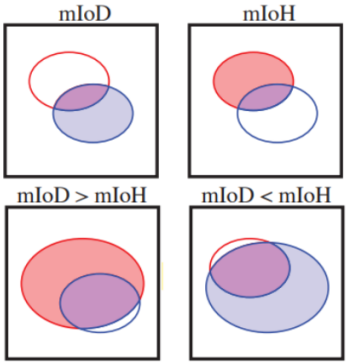
- Study **head-dependent pairs** in sentences using Universal Dependencies (UD) syntax
- For each dependency relation, measure how much the attribution map of the *dependent* overlaps with that of the *head*
- 3 measures: mIoU, mIoD (Mean IoD over the **D**ependent), mIoH (Mean IoD over the **H**ead)

Visuosyntactic Analysis: Head-Dependent Pairs ---



Visuosyntactic Analysis: Measures of Overlap ---

To compare two DAAM maps (e.g. “blue” vs. “teapot”), three overlap metrics:



- **mIoU** — $|A \cap B|/|A \cup B|$: overall similarity between the two maps
- **mIoD** — $|A \cap B|/|A|$: how much of the **dependent’s** map is covered by the head
- **mIoH** — $|A \cap B|/|B|$: how much of the **head’s** map is covered by the dependent

Dominance is measured by the difference $\Delta = \text{mIoD} - \text{mIoH}$:

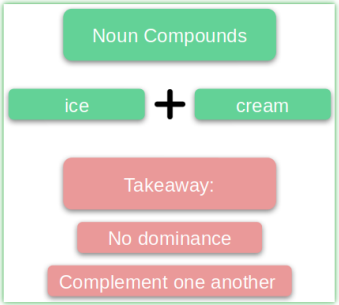
- $\text{mIoD} > \text{mIoH}$ ($\Delta > 0$): the **head dominates** — the dependent’s map is contained within the head’s
- $\text{mIoD} < \text{mIoH}$ ($\Delta < 0$): the **dependent dominates** — the head’s map is contained within the dependent’s

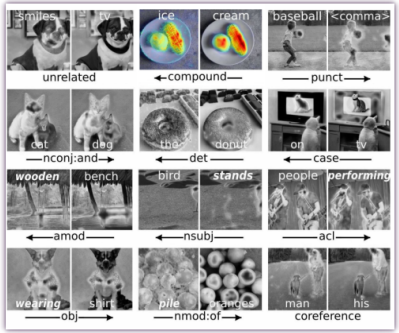
In plain English

In “blue teapot”: does “blue” attend to the same region as “teapot”, or does it spread beyond it? Δ tells you which word’s visual footprint contains the other’s.

Visuosyntactic Analysis: Results Compound ---

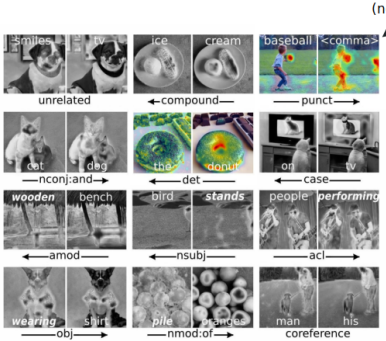
#	Relation	mIoD	mIoH	Δ	mIoU
1	Unrelated pairs	65.1	66.1	1.0	47.5
2	All head–dependent pairs	62.3	62.0	0.3	43.4
3	compound	71.3	71.5	0.2	51.1
4	punct	68.2	70.0	1.8	49.5
5	nconj:and	58.0	56.1	1.9	38.2
6	det	54.8	52.2	2.6	35.0
7	case	51.7	58.1	6.4	36.9
8	acl	**67.4**	79.3	12.	55.4
9	nsubj	76.4	**63.9**	12.	52.2
10	amod	**62.4**	77.6	15.	51.1
11	nmod:of	73.5	**57.9**	16.	47.5
12	obj	75.6	**46.3**	29.	55.4
14	Coreferent word pairs	84.8	77.4	7.4	**66.6**





Visuosyntactic Analysis: Results Punctuation & Articles ---

#	Relation	mIoD	mIoH	Δ	mIoU
1	Unrelated pairs	65.1	66.1	1.0	47.5
2	All head–dependent pairs	62.3	62.0	0.3	43.4
3	compound	71.3	71.5	0.2	51.1
4	punct	68.2	70.0	1.8	49.5
5	nconj:and	58.0	56.1	1.9	38.2
6	det	54.8	52.2	2.6	35.0
7	case	51.7	58.1	6.4	36.9
8	acl	**67.4**	79.3	12.	55.4
9	nsubj	76.4	**63.9**	12.	52.2
10	amod	**62.4**	77.6	15.	51.1
11	nmod:of	73.5	**57.9**	16.	47.5
12	obj	75.6	**46.3**	29.	55.4
14	Coreferent word pairs	84.8	77.4	7.4	**66.6**



Punctuation & Articles

the + donut

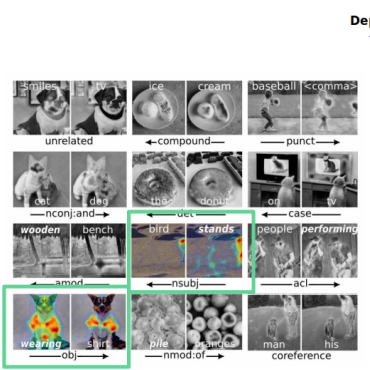
baseball + <comma>

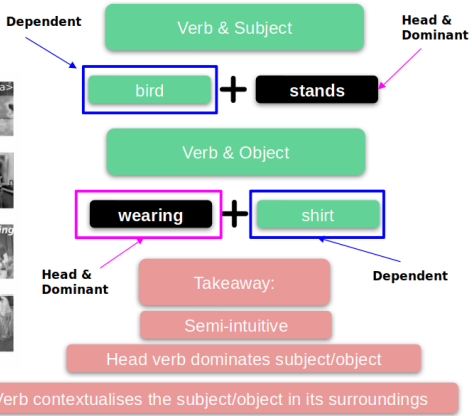
Takeaway:

No dominance

Little semantic meaning

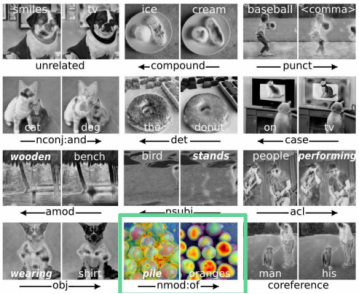
Visuosyntactic Analysis: Results Verb & Subject/Object ---

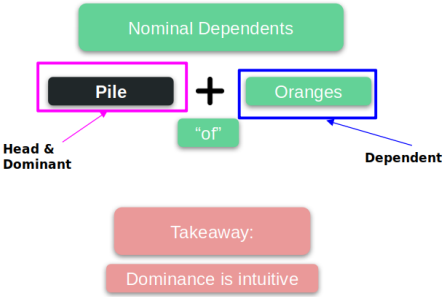




#	Relation	mIoD	mIoH	Δ	mIoU
1	Unrelated pairs	65.1	66.1	1.0	47.5
2	All head–dependent pairs	62.3	62.0	0.3	43.4
3	compound	71.3	71.5	0.2	51.1
4	punct	68.2	70.0	1.8	49.5
5	nconj:and	58.0	56.1	1.9	38.2
6	det	54.8	52.2	2.6	35.0
7	case	51.7	58.1	6.4	36.9
8	acl	**67.4**	79.3	12.	55.4
9	nsubj	76.4	**63.9**	12.	52.2
10	amod	**62.4**	77.6	15.	51.1
11	nmod:of	73.5	**57.9**	16.	47.5
12	obj	75.6	**46.3**	29.	55.4
14	Coreferent word pairs	84.8	77.4	7.4	**66.6**

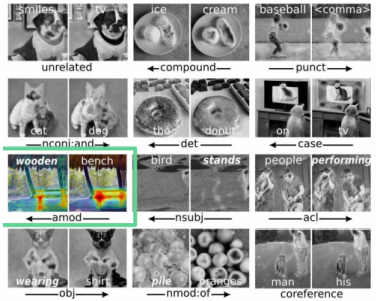
Visuosyntactic Analysis: Results Verb & Subject/Object ---

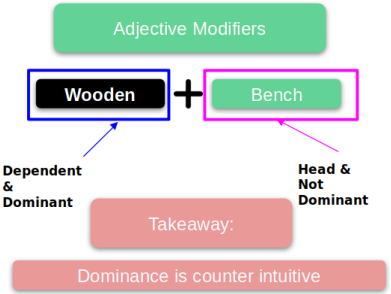




#	Relation	mIoD	mIoH	Δ	mIoU
1	Unrelated pairs	65.1	66.1	1.0	47.5
2	All head–dependent pairs	62.3	62.0	0.3	43.4
3	compound	71.3	71.5	0.2	51.1
4	punct	68.2	70.0	1.8	49.5
5	nconj:and	58.0	56.1	1.9	38.2
6	det	54.8	52.2	2.6	35.0
7	case	51.7	58.1	6.4	36.9
8	acl	**67.4**	79.3	12.	55.4
9	nsubj	76.4	**63.9**	12.	52.2
10	amod	**62.4**	77.6	15.	51.1
11	nmod:of	73.5	**57.9**	16.	47.5
12	obj	75.6	**46.3**	29.	55.4
14	Coreferent word pairs	84.8	77.4	7.4	**66.6**

Visuosyntactic Analysis: Results Adjectives ---





#	Relation	mIoD	mIoH	Δ	mIoU
1	Unrelated pairs	65.1	66.1	1.0	47.5
2	All head–dependent pairs	62.3	62.0	0.3	43.4
3	compound	71.3	71.5	0.2	51.1
4	punct	68.2	70.0	1.8	49.5
5	nconj:and	58.0	56.1	1.9	38.2
6	det	54.8	52.2	2.6	35.0
7	case	51.7	58.1	6.4	36.9
8	acl	**67.4**	79.3	12.	55.4
9	nsubj	76.4	**63.9**	12.	52.2
10	amod	**62.4**	77.6	15.	51.1
11	nmod:of	73.5	**57.9**	16.	47.5
12	obj	75.6	**46.3**	29.	55.4
14	Coreferent word pairs	84.8	77.4	7.4	**66.6**

Visuosyntactic Analysis: Key Takeaways ---

- **Noun Compounds** (*"ice cream"*): No dominance — complement one another
- **Punctuation & Articles** (*the donut*): No dominance — little semantic meaning
- **Verb & Subject** (*bird stands*): Head verb **dominates** — contextualises subject/object in surroundings (*semi-intuitive*)
- **Nominal Dependents** (*pile of oranges*): Head dominates — *intuitive*
- **Adjective Modifiers** (*wooden bench*): Dependent **dominates** — *counter-intuitive*

Summary

Attribution map of the **dependent subsumes** that of the head, and vice versa for others. Dominance is intuitive in some cases but **counter-intuitive** in others.

Visuosemantic Analysis: Cohyponym Entanglement ---

Key Question

Do semantically **similar words** have **worse** generation quality?

Prompt structure: `"a(n) <noun> and a(n) <noun>"`

- **Cohyponym example:** `"a giraffe and a zebra"` — both nouns belong to the same semantic category (animals)
- **Non-cohyponym example:** `"a zebra and a fridge"` — nouns from different semantic categories

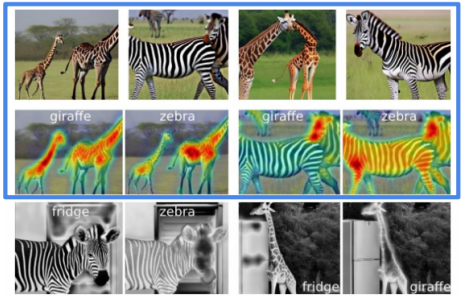
In plain English

Cohyponyms are words that share a common hypernym — "giraffe" and "zebra" are both animals. The hypothesis is that Stable Diffusion struggles to generate *both* objects when they are semantically similar, because their DAAM maps tend to overlap and entangle.

Cohyponym Entanglement: Results ---

Prompt: `"a giraffe and a zebra"`

- Stable Diffusion generation **worsens** with cohyponyms — it generates **one** noun but **not both**
- The DAAM maps of the two nouns **overlap strongly**, revealing **feature entanglement**: the model cannot separate two semantically similar objects in pixel space



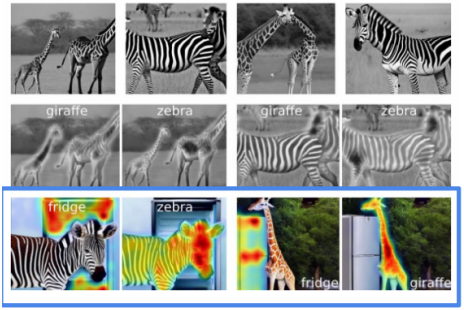
Key Insight

DAAM doesn't just show that generation fails — it explains *why*: when two words attend to the same image regions, the model cannot allocate distinct visual space to each object. Entangled maps are both a symptom and a diagnosis.

Cohyponym Entanglement: Non-Cohyponym Contrast ---

Prompt: `"a zebra and a fridge"`

- Stable Diffusion generates **both** nouns successfully
- The DAAM maps of the two nouns are **distinct** — each word attends to a separate, non-overlapping region



Key Insight

The contrast with the cohyponym case is stark: when two objects are semantically distant, the model allocates **independent visual space** to each — no entanglement, no generation failure. This confirms that overlap in DAAM maps is not accidental but reflects a genuine failure mode of the model.

Visuosemantic Analysis: Adjectival Entanglement ---

Prompt structure: `"<adj> <noun> <verb phrase>"`

- **Examples:**
  - `"a [rusty] shovel sitting in a clean shed"`
  - `"a [bumpy] ball rolling down a hill"`

Expected behavior: if there is **no entanglement**, varying the adjective should only affect the noun's region — the background should **not gain attributes** pertaining to that adjective.

In plain English

If "rusty" only modifies the shovel, swapping it for "shiny" should change the shovel but leave the shed untouched. If the entire image changes, the adjective has leaked beyond its intended target — a sign of feature entanglement.

Adjectival Entanglement: Results ---

Attribution maps for adjectives attend **too broadly** — far beyond the noun they modify. Changing the adjective (rusty → metallic → wooden) alters the **entire scene**, not just the noun. This is **feature entanglement**: the adjective's visual footprint leaks into the background.

Key Insight

DAAM reveals *why* Stable Diffusion struggles with precise attribute binding — adjectives do not attend locally to their noun, but globally to the whole image, making it impossible to change one attribute without affecting the rest of the scene.

Summary ---

- DAAM provides pixel-level attribution maps for Stable Diffusion, a state-of-the-art text-to-image generator
- These maps appear to be informative, as evaluated through segmentation tasks and user study
- DAAM can be a useful tool for further understanding and analyzing Stable Diffusion — e.g. through visuosyntactic analysis and visuosemantic analysis

Class Discussion ---

- Does DAAM give a clear understanding about how a large-scale latent diffusion model synthesizes text to image and which parts of an image are influenced the most?
- Does DAAM explain all the dynamics of how images are synthesized? If not, how should DAAM be modified to better explain image generation?
  - Other explanatory tools besides **attention** and **segmentation proposals**?
- DAAM pointed out failure cases of Stable Diffusion. Are there further interpretability methods needed to understand why **feature entanglement** is occurring and how it could be improved?

Thank You!

References --- I

McGrath, Thomas, Andrei Kapishnikov, Nenad Tomašev, Adam Pearce, Demis Hassabis, Been Kim, Ulrich Paquet, and Vladimir Kramnik. 2022. "Acquisition of Chess Knowledge in AlphaZero." In *Proceedings of the National Academy of Sciences*.

Tang, Raphael, Linqing Liu, Akshat Pandey, Zhiying Jiang, Gefei Yang, Karun Kumar, Pontus Stenetorp, Jimmy Lin, and Ferhan Ture. 2023. "What the DAAM: Interpreting Stable Diffusion Using Cross Attention." In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*.